

Vergleich von Python-Webframeworks Django vs. FastAPI vs. Flask

Sebastian Schöneich

Januar 2026

Die Wahl des optimalen Webframeworks für ein Projekt ist entscheidend für viele Aspekte der Softwarelösung, zum Beispiel Effizienz, Skalierbarkeit und Funktionsumfang. Diese Belegarbeit analysiert und vergleicht die drei gängigsten Python Webframeworks - Django, FastAPI und Flask - generell und am Beispiel einer zu entwickelnden Webanwendung bei dem Praxispartner "SARAD GmbH".

Das Ziel dieser Arbeit ist es das optimale Framework für das Anwendungsbeispiel zu bestimmen. Hierfür werden, für die Softwarelösung, wichtige Aspekte aufgestellt - so zum Beispiel die Komplexität, Performanz und Datenbank-Interaktion - und nach diesen recherchiert und getestet. Nach Recherche und Tests werden die Ergebnisse in einem gewichteten Vergleich gegenübergestellt und ein optimales Framework bestimmt.

Das Ergebnis dieser Arbeit zeigt, dass Flask, aufgrund der minimalistischen, einfachen und modularen Grundherangehensweise, das optimale Framework für das Anwendungsbeispiel ist und für die Erstellung der Software genutzt werden sollte.

Am Ende steht ein Fazit, welches noch einmal kurz über die Eigenschaften jedes Frameworks geht und das Ergebnis erläutert, mit dem Hinweis, dass das Ergebnis einer solchen Analyse stark in Abhängigkeit zum geplanten Softwareprodukt steht.

Duale Hochschule Sachsen - Standort Dresden, SARAD GmbH. Ich bedanke mich bei Komillitonen und Kollgen für Einblicke und Kommentare zur Arbeit und zur Herangehensweise

Contents

1	Einleitung	1
2	Anforderungen	2
3	Frameworks	4
3.1	Django	4
3.1.1	Anwendungsbeispiel	5
3.1.2	Pros und Contras	8
3.2	FastAPI	8
3.2.1	Anwendungsbeispiel	8
3.2.2	Pros und Contras	10
3.3	Flask	10
3.3.1	Anwendungsbeispiel	11
3.3.2	Pros und Contras	11
4	Analyse und Vergleich	12
4.1	Frontend	13
4.2	Datenbank-Integration	13
4.3	Komplexität	14
4.4	API-Schnittstelle	15
4.5	Eigene Einschätzung	15
4.6	Performanz	16
4.7	Erweiterbarkeit	17
4.8	Community	18
4.9	Zusammenfassung	18
5	Fazit	19
6	Quelleangabe	21
7	Hilfsmittel	23

1. Einleitung

Motivation. Mein Praxispartner, die SARAD GmbH, produziert als primären Unternehmensgegenstand Detektoren und Melder für das radioaktive Isotop Radon her, welche in verschiedenen Bereichen, wie z.B. dem Bergbau oder Wohnhäusern, in Gebieten mit hoher Radon-Belastung, eingesetzt werden. Seit ca. 2 Jahren bietet das Unternehmen die Dienstleistung an, dass die, von den einzelnen Geräten gesandten, Daten auf einem zentralen MQTT-Broker gesendet und ebenso zentral verarbeitet werden.

Aufgrund dieser Dienstleistung ist es notwendig Kundendaten ordentlich und übersichtlich vorhanden zu haben um somit eine einfache Möglichkeit zu bekommen Rechnungen zu stellen. Derzeit werden diese Daten in einer unstrukturierten Datenbank, ohne jegliche spezifische Visualisierung, gespeichert und die Rechnungszeiträume müssen händisch angedacht werden. Als Teil meiner Arbeit bei der SARAD GmbH ist es meine Aufgabe eine zentrale Webanwendung zu entwickeln, damit die zuständigen Mitarbeiter einen geordneten Punkt haben um alle Daten visualisiert zu bekommen und eventuelle Verwaltungsaktionen durchzuführen.

Als Student der Medieninformatik ist dieses Thema sehr passend, da die Webentwicklung einer der großen, zentralen Punkte der Medieninformatik darstellt. In diesem Projekt wird es mir also ermöglicht mein Webentwicklungswissen zu vertiefen, Erfahrung für die Praxis zu sammeln und eventuell mein zukünftiges Studium durch Praxiserfahrung zu unterstützen. Ebenso stellt das Projekt eine gute Grundlage für die Erstellung einer Bachelorarbeit dar - und somit diese Arbeit als nützliche Grundlage dafür.

Problemstellung und Zielsetzung. Nachdem eine Bestandsaufnahme getätigt wurde, ist der erste Schritt bei der Erstellung dieser Webanwendung die Frage, welche Technologien ich verwenden möchte oder sollte. Eine zentrale Gruppe der Technologien zur Erstellung von Webanwendungen sind die Webframeworks, hiervon gibt es eine sehr große Anzahl in unterschiedlichen Programmiersprachen. Aufgrund von Aspekten, auf die ich im Verlauf dieser Arbeit eingehen werde, beschränkt sich die Betrachtung der Webframeworks auf jene, die für die Programmiersprache Python entwickelt wurden. Trotz dieser Einschränkung, gibt es weiterhin eine große Anzahl an verschiedenen Webframeworks. Da diese Anzahl den Rahmen dieser Arbeit sprengen würde, werde ich mich auf die drei verbreitetsten Python-Webframeworks beschränken und stelle mir in dieser Arbeit die Frage:

Welches der Python-Webframeworks - Django, FastAPI oder Flask - ist das geeignetste für dieses Projekt?

Um diese Frage zu beantworten werde ich in dieser Arbeit zu erst die zu Grunde liegenden Anforderungen auflisten und bereits gefallene Entscheidungen erläutern, danach eine

Übersicht zu jedem der drei Frameworks bieten, diese grundlegend und an meinem konkreten Anwendungsbeispiel analysieren und vergleichen. Am Ende steht eine, auf Recherchen und, daraus resultierenden, Analysen fundierte Entscheidung welches dieser drei Frameworks für die Entwicklung der oben genannten Webanwendung am besten geeignet ist.

2. Anforderungen

Im folgenden Abschnitt gehe ich auf die verschiedenen Anforderungsaspekte ein und lege den Grundsatz für die Recherche, die Analyse und den Vergleich der Webframeworks. Wie bereits in der Einleitung beschrieben stellt sich zuerst folgende Frage - Warum soll die Webanwendung mit der Programmiersprache Python, anstatt mit einer etablierteren oder spezialisierteren Sprache, wie zum Beispiel PHP oder JavaScript mit Node.js, entwickelt werden? Die Wahl der Programmiersprache resultierte aus drei simplen, aber kritischen Gründen:

Eigene Vorkenntnisse. Ich habe bereits, durch vorherige berufliche und private Projekte, insgesamt über 5 Jahre unterschiedlichste Erfahrungen in der Entwicklung mit Python gesammelt und habe damit die meiste Erfahrung. Die Nutzung von Python reduziert somit die Entwicklungszeit immens, da ich mich nicht in eine mir eher unbekanntere oder gänzlich unbekanntere Sprache einarbeiten muss.

Vorkenntnisse der Mitarbeiter. Die anderen Mitarbeiter der IT- und Entwicklungsabteilung der SARAD GmbH haben ebenso einige Vorkenntnisse in der Nutzung von Python. Somit ist es für diese leichter nachzuvollziehen, wie der Quellcode der Anwendung geschrieben ist und es ist ebenso einfacher möglich bei Bedarf Änderungen zu tätigen, nachdem ich das Unternehmen verlassen habe.

Vorhandene Skripte. Neben der Datenvisualisierung sollen über die Anwendung ebenso Skripte¹ auf einem internen Server ausgeführt werden. Diese Skripte sind, aktuell teilweise - zukünftig vollständig, in Python geschrieben.

Die Wahl der Programmiersprache grenzt zwar bereits die Anzahl der Webframeworks immens ein, dennoch bleibt eine Anzahl von mehr als 50² Webframeworks übrig. Wie bereits in der Einleitung beschrieben, habe ich mich auf die drei Webframeworks Django, FastAPI und Flask begrenzt. Die Gründe hierfür sind:

¹die genannten Skripte werden für die Zertifizierung (Erstellung von Zertifikaten, Rückzug der Zertifikate und Bearbeitung von Berechtigungen) und die Erstellung der Windows-Installationsdateien genutzt. Eine interne Bezeichnung des überstehenden Meta-Skripts ist "MUST - MQTT User Service Tools"

²als Grundlage dieser Schätzung wurde die Anzahl an Python-Frameworks genutzt, die bei den TechEmpower-Benchmarks vorhanden sind.

- **Verbreitung:** Django, FastAPI und Flask sind die aktuelle verbreitetsten Webframeworks für Python. Das macht die Arbeit mit diesen Frameworks sehr viel einfacher, da eine große Anzahl an verschiedenen Quellen, Tutorials oder bereits beantwortete Fragen im Internet gibt. Ebenso gibt es eine Vielzahl von unterschiedlichen Meinungen zu diesen Frameworks, was die Recherche und Evaluation erleichtert.
- **Karrierechancen:** Auch wenn möglicherweise kleinere, besser für meinen Anwendungszweck geeignete Webframework-Projekte im Python-Kosmos existieren, kann ich bei der Arbeit mit Django, FastAPI oder Flask wichtige Arbeitserfahrung sammeln, welche mir zukünftig weitere Karrierechancen ermöglichen können.
- **Vorerfahrungen und Interesse:** In der Vergangenheit konnte ich bereits erste Erfahrungen mit dem Django-Framework sammeln. Im Falle von FastAPI und Flask habe ich zwar noch nicht mit diesen gearbeitet, jedoch ist mein Interesse an diesen beiden Frameworks hoch, da ich bereits Einblicke in Projekte, die mit diesen Frameworks entstanden sind, nehmen.
- **Varianz:** Die drei Kandidaten bieten eine hohe Varianz in ihrer Herangehensweise als Webframework und spiegeln so grundlegende unterschiedliche Archetypen mit unterschiedlichen Fokuspunkten und Philosophien wider.

Um konkrete Gesichtspunkt für eine Recherche und Analyse dieser Webframeworks zu haben, ist es notwendig klare Anforderungen an die Softwarelösung zu definieren.

Aus den Gesprächen mit meinem Betreuer, Herrn Michael Strey, und eigenen Überlegungen wurde folgende Ansprüche an die Softwarelösung gesetzt - sortiert in absteigender Priorität:

- **Visualisierung einer Datenbank:** Die notwendigen Kunden- und Vertragsdaten sollen in einer Datenbank gespeichert werden, die Aufgabe der Software soll es sein diese Datensätze vollständig und tabellarisch im Browser darzustellen. Aufgrund der Art der Daten ist eine relationale Datenbank notwendig³, das bedeutet, dass die Software eine Art des ORM⁴ benötigt.
- **Komplexität:** Da die Softwarelösung rein intern eingesetzt werden soll, soll sie nicht unnötig komplex und groß sein. Bestenfalls sollen nur die wirklich benötigten Funktionen eingebaut werden.
- **Ausführung von Skripten auf einer internen Maschine:** Es soll über die Webanwendung die Möglichkeit bestehen Python-Skripte auf einem internen Server auszuführen. Um diese Funktion zu realisieren über die Webanwendung soll es möglich sein Python-Skripte auf einem internen Server auszuführen. Hierfür wird eine API-Schnittstelle benötigt.

³ zum Zeitpunkt der Verfassung ist noch nicht festgelegt welche Datenbanklösung verwendet wird. Tendenziell wird entweder MariaDB oder SQLite verwendet

⁴ Object-relational mapping

- **Eigene Einschätzung:** Da ich der einzige Entwickler dieser Softwarelösung sein werde, ist meine eigene Einschätzung der Frameworks, wie passend sie mit ihren Eigenschaften für dieses Projekt sind, ein wichtiger Gesichtspunkt. Ebenso ist meine eigenes Interesse an der Arbeit mit den Frameworks relevant.
- **Performanz:** Die Menge der Nutzer dieser Software wird sich auf insgesamt maximal 4-5 Personen beziehen. Dementsprechend ist auszugehen, dass nur etwa 1-2 Anfragen gleichzeitig stattfinden werden. Diese Tatsache macht den Aspekt der Performanz, zwar deutlich unwichtiger als er sonst bei Webanwendungen ist, jedoch ist es immer noch ein zu beachtender Punkt.
- **Erweiterbarkeit:** Da sich Software und die Anforderungen an diese andauernd verändern, soll es Möglichkeiten geben die Webanwendung zukünftig einfach zu erweitern, falls sich neue Anforderungen an das Produkt ergeben sollten.

3. Frameworks

Im folgenden Abschnitt gehe ich auf die einzelnen Frameworks ein, dabei werde ich auf allgemeine Informationen zum Framework an sich, die grundlegende Philosophie und die Schritte zu einer simplen "Hello World"-Website⁵ eingehen. Nach jedem Beispiel steht eine kleine Aufstellung von Aspekten die für oder gegen das Framework sprechen⁶.

3.1. Django

Django, ursprünglich entwickelt von Adrian Holovaty und Simon Wilson, wurde im July 2005 veröffentlicht und ist damit das älteste der drei Frameworks, die sich im Vergleich befinden. Aktuell befindet sich Django in der Version 6.0⁷ und wird heutzutage von der Django Software Foundation⁸ aktuell gehalten.

Das Framework ist vollständig Open-Source und ist zu etwa 97% in Python⁹ geschrieben. Das Framework wird als *"The Webframework for perfectionists with deadlines"* beschrieben und verfolgt als Full-Stack-Lösung eine *"batteries included"*-Philosophie, was so viel bedeutet, wie, dass Django grundsätzlich mit allen Modulen und Funktionen, die eine Webanwendung benötigt kommt.

Unter diesen Modulen und Funktionen befinden sich zum Beispiel:

- ein Entwicklungs-Webserver für Tests

⁵einfachste anzunehmende Anwendung - die Software gibt einfach nur den Text "Hello World" zurück

⁶diese Aspekte sind aus eigener Erfahrung entstanden. Es wurde jeweils etwa 5 Stunden mit jedem Framework gearbeitet um diese Aspekte benennen zu können.

⁷am 3. Dezember 2025 veröffentlicht

⁸NonProfit-Organisation, zur Entwicklung und Instandhaltung von Django

⁹siehe <https://github.com/django/django>

- ein Templating-System
- ein Caching-Framework
- Middleware-Support
- ein Internationalisierungs-System
- ein Authentifikations-System
- ein dynamisches Admin-Interface

Zudem ist Django auch noch mit Drittanbieter-Paketen erweiterbar.

3.1.1. Anwendungsbeispiel

Django liegt eine grundlegende Unterscheidung zu Grunde - **Projekte und Apps**.

Eine App ist eine vollständige Web-Applikation, die etwas ausführt. Apps können zum Beispiel eine Umfrage, ein Blog oder die angeforderte Softwarelösung sein.

Ein Projekt ist eine Sammlung von Konfigurationen und Apps für eine bestimmte Website. Bei diesem System gilt, dass ein Projekt mehrere Apps beinhalten kann und eine App in mehreren Projekten vorhanden sein kann.

Im Falle dieses Anwendungsbeispiels ist ein Projekt und eine App notwendig¹⁰.

Im Nachfolgendem werden die Schritte¹¹ beschrieben um ein einfaches Django-Projekt mit einer "Hello World"-App aufzusetzen:

a. **Installation mittels pip**¹²

Folgender Befehl wird ausgeführt:

```
python -m pip install Django
```

b. **Projekt-Verzeichnis erstellen**

Folgender Befehl wird ausgeführt:

```
mkdir django
```

c. **Projekt initialisieren**

Folgender Befehl wird ausgeführt:

```
django-admin startproject project django
```

Nach diesem Shell-Befehl entsteht im django-Verzeichnis folgende Dateistruktur:

¹⁰ selbiges würde auch für die geplante Softwarelösung gelten.

¹¹ alle hier beschriebenen Aktionen wurden in einer Linux-Distribution durchgeführt, falls man diese Schritte mit einem anderen Betriebssystem durchführen möchte, muss man die entsprechenden Befehlsäquivalente benutzen

¹² Paketmanager von Python

```
django
  manage.py
  project/
    __init__.py
    asgi.py
    settings.py
    urls.py
    wsgi.py
    __pycache__/  
    ...
```

d. **App erstellen**

Zuerst muss das Arbeitsverzeichnis gewechselt werden:

```
cd django
```

Dann muss folgender Befehl ausgeführt werden:

```
python manage.py startapp myapp
```

Nach diesem shell-Befehl entsteht im `django`-directory folgende zusätzliche Dateistruktur:

```
django
...
myapp/
  __init__.py
  admin.py
  apps.py
  models.py
  tests.py
  views.py
  migrations/
    __init__.py
```

e. **View erstellen und per URL aufrufbar machen**

In der `myapp/views.py`-Datei wird folgender Code eingefügt:

```
from django.http import HttpResponse
```



```
def index(request):  
    return HttpResponse("Hello World")
```

Um diesen Inhalt per URL aufrufbar machen zu können muss eine entsprechende URL-Konfiguration erstellt werden. Diese sind in der Django-Umgebung Python-Dateien mit dem Namen `urls.py`.

In diesem Fall muss eine `urls.py`-Datei im `myapp/`-Verzeichnis erstellt werden. Sobald die Datei erstellt wurde, wird folgender Code in sie eingefügt:

```
from django.urls import path  
from . import views  
  
urlpatterns = [  
    path("", views.index, name="index")  
]
```

Als letzten Schritt muss nun noch die URL-Konfiguration des Projektes an sich korrekt gesetzt werden. Hierfür schreibt man folgenden Code in die Datei `django/project/urls.py`:

```
from django.contrib import admin  
from django.urls import include, path  
  
urlpatterns = [  
    path("myapp", include("myapp.urls")),  
    path("admin", admin.site.urls)  
]
```

f. Testaufruf

Um den Entwicklungsserver von Django zu starten, und somit die "Hello World"-Seite im Browser aufrufbar machen zu können gibt man folgenden shell-Befehl im `django`-Verzeichnis ein:

```
python manage.py runserver
```

Der Server ist nun hochgefahren und beim Aufruf der Seite `http://localhost:8000/myapp/` öffnet sich folgende Website:

3.1.2. Pros und Contras

Die offensichtlichen Vorteile von Django sind klar - **es wird sofort fast¹³ alles mitgeliefert**, was bei einer Webapplikation benötigt werden könnte, und es wird eine **klare Dateistruktur** erstellt, die bei jedem einzelnen Django-Projekt gleich ist, was speziell bei zukünftiger, erneuter Arbeit mit Django ein Vorteil ist, es ist jedoch **schwer sich ersteinmal überhaupt in diese Struktur einzuarbeiten**. Zusätzlich zu der Struktur, kommt der, für Python-Verhältnisse, **komplexe Quellcode** und der **große Konfigurationsaufwand**. Einen großen Teil¹⁴ bei der Einarbeitungszeit mit Django wurde dafür verwendet in der Dokumentation nachzuschlagen.

3.2. FastAPI

FastAPI, entwickelt und instandgehalten von Sebastián "tiangolo" Ramírez, wurde im Dezember 2018 veröffentlicht und ist damit das jüngste der Framework-Kandidaten. Aktuell befindet sich FastAPI in der Version 0.124.4¹⁵ und ist ein Open-Source API-Framework, welches zu 100% in Python¹⁶ geschrieben ist. FastAPI ist eine ausschließliche Backend-Lösung für APIs und hat somit keine Frontendfunktionen, wie einen Template-Renderer. Im Kern und der Namensgebung von FastAPI befindet sich die Geschwindigkeit des Frameworks, welche durch die grundlegend asynchrone Funktionabilität entsteht.

3.2.1. Anwendungsbeispiel

a. FastAPI und Uvicorn installieren

FastAPI kann mittels pip installiert werden. Da FastAPI keinen integrierten Entwicklungssever hat. Um FastAPI zu starten wird der ASGI-Server "uvicorn" benötigt. Es wird folgender Befehl ausgeführt:

```
python pip install fastapi uvicorn
```

b. Projekt-Verzeichnis erstellen und Arbeitsverzeichnis wechseln

Folgende Befehle wird ausgeführt:

```
mkdir fastapi  
cd fastapi
```

¹³ eine API-Schnittstelle fehlt und muss mit einem Community-Paketen importiert werden - auf diesen Aspekt wird im Abschnitt "Vergleich und Analyse" eingegangen

¹⁴ geschätzt 40% der Zeit

¹⁵ am 12. Dezember 2025 veröffentlicht

¹⁶ siehe <https://github.com/fastapi/fastapi>

c. **Hello World-Code schreiben**

Folgender Code wird in eine neu erstellte 'main.py'-Datei geschrieben:

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def index():
    return {"message": "Hello World"}

```

d. **uvicorn-Server starten**

Folgender Befehl wird ausgeführt:

```
uvicorn main:app --reload
```

e. **Testaufruf**

Der Server ist nun hochgefahren und beim Aufruf der Seite `http://localhost:8000` öffnet sich folgende Website:

3.2.2. Pros und Contras

FastAPI, im direkten Vergleich zu Django hat nahezu keinen Konfigurationsaufwand und hat eine sehr kleine Dateistruktur. Der Quellcode ist aufgrund meiner Vorerfahrungen mit Asynchronität in Python¹⁷ sehr leicht verständlich. Auch die komplett automatisch erzeugte OpenAI¹⁸ Dokumentation ist sehr nützlich, jedoch gibt es ein konkretes, großes Problem - FastAPI hat keine Frontend-Funktionabilitäten, hierfür würde weiteres Framework wie React benötigt.

3.3. Flask

Das dritte und letzte Framework in der Auswahl, Flask, wurde am 1. April 2010 ursprünglich als Aprilscherz von Armin Ronacher veröffentlicht. Aktuell befindet sich Flask in der Version 3.1.2.¹⁹ und wird heutzutage von der Pallets-Organisation²⁰ erweitert und instand gehalten. Das Framework ist zu 100% in Python geschrieben und vollständig Open Source. Wie Django, Flask ist eine FullStack-Lösung, besitzt jedoch eine grundlegend gegensätzliche Philosophie - Minimalismus. Flask wird als Micro-Framework bezeichnet und kommt an sich mit sehr wenigen Komponenten, das Grundframework kann jedoch mit vielen verschiedenen Communit-Paketen erweitert werden.

¹⁷Vorerfahrungen entstanden hauptsächlich bei der Arbeit mit discord.py - eine Schnittstelle zur Interaktion mit Bots auf der Kommunikationsplattform Discord

¹⁸ehemals Swagger

¹⁹am 19.08.2025 veröffentlicht

²⁰Organisation, welche beliebte und weiter verbreitete Python-Frameworks weiterentwickelt und instand hält, z.B. Jinja - ein Template-Framework

3.3.1. Anwendungsbeispiel

a. Flask installieren

Flask kann mittels pip installiert werden. Es wird folgender Befehl ausgeführt:

```
python pip install Flask
```

b. Projektverzeichnis erstellen und Arbeitsverzeichnis wechseln

Folgende Befehle werden ausgeführt:

```
mkdir flask  
cd flask
```

c. Hello World-Code schreiben

Folgender Code wird in eine neu erstellte `app.py`-Datei geschrieben:

```
from flask import Flask  
  
app = Flask(__name__)  
  
@app.route("/")  
def hello():  
    return "Hello World"
```

d. Testserver starten

Es wird folgender Befehl ausgeführt:

```
flask run
```

e. Testaufruf

Der Server ist nun hochgefahren und beim Aufruf der Seite `http://localhost:5000` öffnet sich folgende Website:

3.3.2. Pros und Contras

Flask bietet die Simplizität von FastAPI mit dem Funktionsumfang von Django, wenn man sich die entsprechenden Pakete hinzuzieht, die man für sein Projekt benötigt. Für die notwendigen SQL-Operationen benötigt es zum Beispiel eine Erweiterung²¹, eine Sache,

²¹genutzt wurde flask-sqlalchemy

die sich bei der Arbeit mit Flask als roter Faden durchzieht. Ein potentielles Problem ist, dass viele Erweiterungen von Flask von unterschiedlichsten Entwicklern erstellt werden, weswegen eventuell Konflikte entstehen können - ich konnte jedoch keine solche in meiner Einarbeitung feststellen.

4. Analyse und Vergleich

Im nachfolgenden Teil gehe ich auf jedes Kriterium genau ein, und gebe jedem Framework in dem Kriterium eine Punktzahl zwischen 0 und 5 Punkten, wobei 0 bedeutet, dass das Framework das Kriterium gar nicht erfüllt und 5 bedeutet, dass das Kriterium bestens erfüllt wird. Jedes Kriterium erhält ebenso eine tabellarische Übersicht zur Punkteverteilung mit Anmerkungen.

Für die konkrete Evaluation und den Vergleich der drei Webframeworks, benötigt man Kriterien. Da bereits in den Anforderungen die wichtigsten Gesichtspunkte grundlegend genannt wurden, müssen diese jetzt konkretisiert werden.

Kriterium	Erläuterung	Wichtung
Frontend	Hat dieses FW eine Frontend-Komponente	2x
Datenbank-Integration	Beschreibt ob das FW eine eingebaute Datenbank-Integration hat oder wie schwer es ist eine zu integrieren	2x
Komplexität	Beschreibt wie hoch die Komplexität des FW ist und wie steil die Lernkurve ist um mit diesem FW zu arbeiten	1,5x
API-Schnittstelle	Beschreibt in wie weit API-Schnittstellen mit dem FW erstellt werden können und die Komplexität des Aufsetzens dahinter	1,5x
Eigene Einschätzung	Beschreibt meine eigene Einschätzung des FW in unterschiedlichen Aspekten, z.B. Präferenz, Vorerfahrungen und Interesse	1x
Performance	Beschreibt die Performance des FW im Bezug auf Anfragen und Inhaltsrendering	1x
Erweiterbarkeit	Beschreibt wie viele Erweiterungsmöglichkeiten das FW besitzt und wie es skaliert werden kann	1x
Community	Beschreibt wie gut die Community etabliert ist und wie leicht man Antworten auf seine Fragen bekommt	1x

TABLE 1. Evaluationskriterien und deren Wichtung

4.1. Frontend

Da die Softwarelösung eine visuelle Webanwendung sein soll, ist eine Frontend-Komponente absolut unverzichtbar. Sowohl Django als auch Flask kommen in ihrer Standardfassung mit einer vollständigen Frontend-Engine mit eingebautem Template-Rendering, das bedeutet, dass HTML-Dateien als Templates mit entsprechenden Variablen-Feldern gespeichert werden. Beim Aufruf der Website werden die Template-Variablen mit entsprechenden Inhalten ersetzt und somit eine Website mit den entsprechenden Daten angezeigt. Somit sind alle Anforderungen erfüllt und es werden 5 von 5 Punkten vergeben.

FastAPI ist von Grund auf ein reines API-Framework, besitzt also KEINE Frontend-Engine. um entsprechende Frontend-Inhalte zu produzieren würde es ein weiteres Framework benötigen. Da in dieser Analyse ausschließlich auf die drei genannten Frameworks geschaut wird, müssen 0 von 5 Punkten vergeben werden.

Framework	Bewertung	Anmerkung
Django	5/5	Vollständige Frontend-Engine mit Template Rendering
FastAPI	0/5	KEINE Frontend-Engine
Flask	5/5	Vollständige Frontend-Engine mit Template Rendering

TABLE 2. Bewertungskriterium: Frontend

4.2. Datenbank-Integration

Um die richtigen Inhalte an das Template-Rendering zu übergeben benötigt es eine entsprechende Datenbank-Integration. Weiter oben wurde bereits festgelegt, dass aufgrund der Art der Daten eine relationale Datenbank notwendig ist, was automatisch bedeutet, dass ein entsprechendes ORM vonnöten ist.

Django hat out-of-the-box ein integriertes ORM, somit können volle 5 von 5 Punkten vergeben werden. Sowohl FastAPI als auch Flask kommen nicht direkt mit einem integrierten ORM, jedoch gibt es für beide Frameworks eine konkrete Lösung: Bei FastAPI wird empfohlen SQLAlchemy zu nutzen, ein ORM, welches vom selben Entwickler erschaffen wurde. Im Falle von Flask existiert ein Modul mit dem Namen "Flask-SQLAlchemy", welches ein konkretes Flask-Modul des weit verbreiteten Python-Paket "SQLAlchemy"²² ist. Aufgrund der bereits etablierten Modularität von Flask, habe ich mich dafür entschieden Flask in diesem Punkt etwas höher als FastAPI zu bewerten. Dementsprechend erhält FastAPI 3.5 von 5 Punkten und Flask 4 von 5 Punkten.

²²dieses Paket beinhaltet sowohl ein ORM, als auch ein Toolkit zur Arbeit mit relationalen Daten

Framework	Bewertung	Anmerkung
Django	5/5	integriertes ORM
FastAPI	3,5/5	nicht integriert, aber gute Zusammenarbeit mit SQLAlchemy
Flask	4/5	nicht integriert, aber konkretes Flask-Modul für SQLAlchemy

TABLE 3. Bewertungskriterium: Datenbank-Integration

4.3. Komplexität

Da ich mit keinem der drei Frameworks viel Erfahrung sammeln konnte, ist der Aspekt der Komplexität und Lernkurve relevant, damit ich schnellstmöglich ordentlich mit diesen arbeiten und die Webanwendung entwickeln kann.

Django hat aufgrund der komplexen Dateistruktur, dem Konfigurationsaufwand und hohen Anzahl von integrierten Modulen, eine sehr hohe Einstiegshürde²³. Deswegen erhält Django 2 von 5 Punkten. Sowohl FastAPI als auch Flask haben einfache Dateistruktur und wenige Module "out-of-the-box", was den Einstieg sehr vereinfacht. Häufig wird als Einstiegsschwierigkeit bei diesen beiden Frameworks das Arbeiten mit Asynchronität genannt - diesen Punkt werde ich jedoch außenvor lassen, da ich bereits viel Erfahrungen mit dem Entwickeln mit Asynchronität gesammelt habe und dies somit keine Hürde mehr darstellt. FastAPI erhält deswegen 5 von 5 Punkten. Im Fall von Flask ziehe ich jedoch einen Punkt ab, da die Modulauswahl und -nutzung am Anfang für Probleme sorgen kann - somit landet Flask bei 4 von 5 Punkten.

Framework	Bewertung	Anmerkung
Django	2/5	Komplexe Dateistruktur, viele integrierte Module
FastAPI	5/5	Einfache Dateistruktur, ausschließlich API dementsprechend wenige Module
Flask	4/5	Einfache Dateistruktur, wenige Grundmodule, eventuelle Schwierigkeiten beim Heraussuchen der benötigten Module

TABLE 4. Bewertungskriterium: Komplexität

²³die offizielle Django-Dokumentation besitzt, abseits von How-To und FAQ-Punkten, 270 Punkte und Unterpunkte

4.4. API-Schnittstelle

Um die Ausführung der Skripte zu gewährleisten, ist es notwendig per API-Call einen Befehl an die entsprechende Maschine zu senden. Dementsprechend ist eine API-Schnittstelle notwendig um das durchzuführen.

Django hat kein integriertes API-Framework. Hierfür benötigt man das "Django REST framework", welches man sich separat installieren muss, ebenso ist diese Software nicht vom offiziellen Django-Entwicklerteam. Django erhält in diesem Aspekt 3 von 5 Punkten - einen Punkt Abzug dafür, dass das REST-Framework nicht integriert ist und einen weiteren, da es fragwürdig ist, dass eine so zentrale Funktion eines Frameworks bei einer "batteries included"-Philosophie fehlt. FastAPI ist ein reines API-Framework, dementsprechend erhält es 5 von 5 Punkten. Flask benötigt für API-Calls ein Modul, "FlaskRESTful", welches nicht standardmäßig integriert ist. Flask erhält deswegen 4 von 5 Punkten.

Framework	Bewertung	Anmerkung
Django	2/5	Nicht integriert - "Django REST framework" benötigt
FastAPI	5/5	Integriert
Flask	4/5	Nicht integriert - "FlaskRESTful" benötigt

TABLE 5. Bewertungskriterium: API-Schnittstelle

4.5. Eigene Einschätzung

Da ich selber mit einem der Frameworks arbeiten muss, ist es ebenso relevant wie meine Einstellung zu jedem ist.

Ich habe bereits in zwei kleineren Projekten mit Django gearbeitet, dennoch habe nicht nicht ansatzweise ausreichend Erfahrungen sammeln können um einen Vorteil zu haben. Die hohe Komplexität, das hohe Konfigurationsmanagement und die riesige Anzahl an Modulen sorgt dafür, dass ich Django in diesem Projekt als "Overkill" erachte, das bedeutet, dass das, was Django bietet schlichtweg zu viel ist und gar nicht benötigt wird. Aufgrund dieser Aspekte erhält Django insgesamt 2 von 5 Punkten.

Die Entwicklung von FastAPI war über die letzten Jahre für mich immer sehr interessant zu verfolgen - nicht zuletzt, da auch Freunde und Bekannte intensiv mit diesem Framework arbeiten. Die Performanz und Simplität dieses Frameworks lässt mich vermuten, dass es zukünftig eine größere Rolle spielen wird. Das größte Problem, das FastAPI hat, ist, dass es ausschließlich eine Backend-Engine ist, Für dieses Projekt ist das sehr ungünstig, da ich der einzige Entwickler bin und ich somit noch ein zweites Framework, z.B. React, benötigen würde um den Frontend-Teil zu realisieren. FastAPI erhält deswegen 3 von 5 Punkten.

Anfangs war Flask simplerweise ein dritter Kandidat, jedoch hat sich diese Betrachtung über die Arbeit hinweg stark geändert. Die grundlegende Einarbeitung und Flask war sehr interessant für mich und die grundlegende Modularität macht es einfach Projekte nach Bedarf zu erweitern und nur das zu nutzen, was auch wirklich benötigt wird. Häufig wird die Performanz von Flask als Schwäche genannt, da diese sehr abhängig von den genutzten Modulen ist, da dieses Projekt nur wenige Module nutzen wird, sollte das jedoch kein Problem darstellen. Flask erhält deswegen die vollen 5 Punkte.

Framework	Bewertung	Anmerkung
Django	2/5	Vorerfahrungen (aber zu wenige), sehr komplex und anstrengend, geringes Interesse, Overkill
FastAPI	5/5	sehr hohes Interesse und vermutete Zukunftsrelevanz, leider suboptimal für dieses Projekt
Flask	4/5	hohes Interesse, einfache Einarbeitung, ausreichend leistungsfähig für dieses Projekt

TABLE 6. Bewertungskriterium: Eigene Einschätzung

4.6. Performanz

Auch wenn Performanz in diesem Projekt eher eine nebengestellte Rolle spielt, ist sie eine der besten Aspekte eines Frameworks um konkrete Vergleiche zu machen. Die Benchmarks beziehe ich von TechEmpower's Round 21²⁴, da dies das letzte Benchmark von TechEmpower ist, wo alle drei Kandidaten vorhanden sind. Von jedem Testtypen werden die "Best"-Tabellen hergezogen, insofern sie vorhanden sind:

- bei den Tests "Multiple queries" und "Data updates" wird die "20-queries"-Tabelle benutzt
- beim "Cached queries"-Test gibt es keine verfügbaren Daten
- für Flask wurde "flask-raw" genutzt, da "flask" nicht in allen Tests angetreten ist
- für Django wurde "django" genutzt
- für FastAPI wurde "fastapi" genutzt

²⁴vom 19.07.2022

Test	Django	FastAPI	Flask	Gewinner
JSON serialization	73.479	154.061	91.225	FastAPI
Single query	19.402	62.020	44.033	FastAPI
Multiple queries	1.396	12.406	13.574	Flask
Fortunes	15.038	50.608	36.243	FastAPI
Data updates	754	5.803	8.813	Flask
Plaintext	79.188	155.667	101.348	FastAPI

TABLE 7. TechEmpower Round21 Benchmark Ergebnisse

Anhand dieser Tabelle sieht man, dass FastAPI in fast allen Fällen mit Abstand die beste Performanz bietet und Django überall die schlechteste, teilweise nur 1/8 der Performanz von FastAPI. Die Ergebnisse von Flask sind generell etwas skeptisch zu betrachten, da die Performanz von Flask sehr stark davon abhängig ist welche und wie viele Module benutzt werden. Es ist aber davon auszugehen, dass Flask in vielen Fällen eine bessere Performanz bietet als Django. Ein weiterer, wichtiger, performanzbezogener Gesichtspunkt ist, dass Django standardmäßig synchron arbeitet, FastAPI und Flask jedoch asynchron, wodurch mehrere Anfragen gleichzeitig besser bearbeitet werden können. Bezugnehmend auf alle diese Aspekte bekommt Django 2 von 5 Punkten, FastAPI 5 von 5 Punkten und Flask 3,5 von 5 Punkten.

Framework	Bewertung	Anmerkung
Django	2/5	gute Performanz, synchron
FastAPI	5/5	sehr gute Performanz, asynchron
Flask	3,5/5	Performanz abhängig von genutzten Modulen, asynchron

TABLE 8. Bewertungskriterium: Performanz

4.7. Erweiterbarkeit

Zwar ist die Erweiter- und Skalierbarkeit in diesem Projekt nicht von großer Bedeutung, aber für die allgemeine Betrachtung der Frameworks. Es ist wichtig zu erwähnen, dass eine vertikale Skalierung²⁵ immer eine Grenze hat, die sich am aktuellen Technologiestand orientiert, eine horizontale Skalierung²⁶ keine²⁷ Grenzen besitzt, wenn man finanzielle

²⁵vertikale Skalierung kann man vereinfacht als Erhöhung Rechenleistung der bereits vorhandenen Maschinen beschreiben

²⁶horizontale Skalierung kann man vereinfacht als Erhöhung der Anzahl benutzter Maschinen beschreiben

²⁷es gibt theoretische Grenzen, z.B. gesamt verfügbare Ressourcen auf dem Planeten Erde, jedoch sind diese praktisch nicht relevant und für ein Unternehmen real erreichbar

Aspekte ignoriert.

Django, mit der "batteries included"-Philosophie, kommt als Gesamtpaket, das bedeutet, dass eine Erweiterung in vielen Fällen nicht notwendig ist, jedoch gibt es einige Möglichkeiten zur Erweiterung²⁸. Aufgrund der primär synchronen Arbeitsweise von Django, ist eine vertikale Skalierung eher weniger von Relevanz, da die Arbeitslast nicht asynchron vom gleichen Prozess bearbeitet werden kann, sondern per Loadbalancer an verschiedene Prozesse, Prozessoren oder gar Maschinen verteilt werden muss. Django erhält damit 3 von 5 Punkten. FastAPI hat ebenso einige Erweiterungsmöglichkeiten²⁹, kommt aber schnell an seine Limitationen³⁰. Bezüglich Skalierung ist es bei FastAPI, aufgrund der asynchronen Arbeitsweise, deutlich leichter vertikal zu skalieren. Damit erhält FastAPI ebenso 3 von 5 Punkten. Flask hat als Kern eine modulare Philosophie und ist damit sehr leicht nach Bedarf erweiterbar. Es besteht jedoch Achtung, da die Anzahl und Art von Modulen die Leistung des Frameworks beeinflussen können. Wie bereits bei FastAPI, sorgt die asynchrone Arbeitsweise von Flask dafür, dass vertikale Skalierung sinnvoller zu realisieren ist. Flask erhält damit 5 von 5 Punkten.

Framework	Bewertung	Anmerkung
Django	3/5	einige Erweiterungen möglich
FastAPI	3/5	einige Erweiterungen, jedoch schnell Limitationen, gut vertikal skalierbar
Flask	5/5	sehr viele Erweiterungen möglich, können aber Performanz beeinflussen, gut vertikal skalierbar

TABLE 9. Bewertungskriterium: Erweiterbarkeit

4.8. Community

Alle drei Frameworks haben eine große Community, viele Ressourcen und zahlreiche Möglichkeiten Antworten auf seine Fragen zu bekommen. Deswegen erhalten alle 5 von 5 Punkten.

4.9. Zusammenfassung

Es folgt eine Übersicht über alle oben genannten Aspekte und die vergebenen Punkte:

²⁸ siehe Abschnitt "API-Schnittstelle"

²⁹ siehe Abschnitt "Datenbank-Integration"

³⁰ siehe Abschnitt "Frontend"

Bewertungskriterium	Django	FastAPI	Flask
Frontend	10	0	10
Datenbank-Integration	10	7	8
Komplexität	3	7,5	6
API-Schnittstelle	4,5	7,5	6
Eigene Einschätzung	2	3	5
Performanz	2	5	3,5
Erweiterbarkeit	3	3	5
Community	5	5	5
Summe	39,5	38	48,5

TABLE 10. Zusammenfassung Bewertungskriterien

FastAPI belegt in diesem Vergleich den letzten Platz, was eindeutig auf die Notwendigkeit der Frontend-Engine zurückzuführen ist, die FastAPI nicht bietet. Würde man diesen Aspekt ausblenden, würde FastAPI einen sehr starken zweiten Platz belegen. Django ist in den Aspekten Frontend und Datenbank-Integration die beste Wahl, jedoch fällt die fehlende integrierte API-Schnittstelle, die Komplexität dieses Frameworks und schwache Performanz stark negativ auf. Das alles sorgte dafür, dass Django auf den letzten Platz landet. Flask zeigt sich durch die immense Flexibilität und Simplizität, die es bietet, als Sieger dieses Vergleichs und damit als das passendste Framework für dieses Projekt.

5. Fazit

Alle verglichenen Frameworks haben, wie in dieser Arbeit ausführlich beschrieben, ihre Vor- und Nachteile und eignen sich für unterschiedliche Projekte. Dementsprechend ist das Ergebnis nicht als allgemeingültig zu sehen. **Django** ist aufgrund seiner grundlegenden Struktur und beigelieferten Module eine gute Lösung für stabile, große Projekte, wo bestenfalls verschiedene, selbst entwickelte, Module vermehrt eingesetzt werden können, um das volle Potential der Apps-Projecs-Unterscheidung auszuschöpfen, jedoch benötigt man viel Erfahrung um mit diesem Framework schnell und produktiv arbeiten zu können, anstatt andauernd in der Dokumentation nachzulesen. **FastAPI** lädt dazu ein, eine klare Unterteilung von Entwicklern in Frontend und Backend zu tätigen, und legt einen klaren und deutlichen Fokus auf Geschwindigkeit und Performanz der APIs. Es ist am besten für Projekte geeignet, wo diese Aspekte einen wichtigen Aspekt darstellen. **Flask** ist am ehesten für kleinere Projekte geeignet, die nicht alle Module benötigen, die Django mitliefert. Das Framework hat ebenso eine niedrige Einstiegshürde und eignet sich dementsprechend für alle Entwickler, unabhängig vom Erfahrungsstand.

Im Fall des spezifizierten Softwareproduktes eignet sich, wie man aus der Analyse

herauslesen kann, Flask am deutlichsten, da sich die Vorteile von Flask mit den Anforderungen decken und die Nachteile des Frameworks nahezu irrelevant sind.

6. Quelleangabe

Offizielle Dokumentationen:

- Django: <https://docs.djangoproject.com/en/5.0/>
(Letzter Aufruf: 08.01.2026 9:58)
- FastAPI: <https://fastapi.tiangolo.com/de/>
(Letzter Aufruf: 08.01.2026 10:00)
- Flask: <https://flask.palletsprojects.com/en/stable/>
(Letzter Aufruf: 09.01.2026 13:42)

Repositories

- Django: <https://github.com/django/django>
(Letzter Aufruf: 18.12.2025 16:53)
- FastAPI <https://github.com/fastapi/fastapi>
(Letzter Aufruf: 19.12.2025 13:27)
- Flask <https://github.com/pallets/flask>
(Letzter Aufruf: 07.01.2026 10:38)

Benchmarks:

- TechEmpower: <https://www.techempower.com/benchmarks>
(Letzter Aufruf: 05.01.2026 12:36)

Sonstige Quellen:

- GeeksForGeeks: <https://www.geeksforgeeks.org/python/comparison-of-fastapi-with-django-and-flask/>
(Letzter Aufruf: 08.12.2025 10)
- Medium - Kapil Khatik: <https://medium.com/@kapildevkhatik2/django-vs-flask-choosing-the-right-web-framework-b8857ae38c67>
(Letzter Aufruf: 04.12.2025 11:37)
- Medium - Sai Nitesh Palamakula: <https://medium.com/@sainitesh/flask-vs-django-vs-fastapi-a-beginners-guide-to-choosing-the-right-python-web-framework-e5dd888a3495>
(Letzter Aufruf: 04.12.2025 10:33)
- Medium - Tom Harrison: <https://tomharrisonjr.medium.com/flask-vs-django-why-i-chose-django-d248066ddd45>
(Letzter Aufruf: 04.12.2025 10:59)
- Medium - Viraj Sharma: <https://medium.com/wetheitguys/django-vs-flask-comparison-and-basic-project-setup-0b8e37589b19>
(Letzter Aufruf: 04.12.2025 10:44)
- Medium - Anusha: <https://medium.com/@athatikonda12/%EF%B8%8F-flask-vs-django-vs-fastapi-pros-cons-use-cases-9d60ca078962>
(Letzter Aufruf: 04.12.2025 10:21)

- Medium - Anas Issath: <https://medium.com/@anas-issath/6a8d9741e7ee?sk=b064e7d3f4e01746acf6f07d2d9a8838>
(Letzter Aufruf: 04.12.2025 11:09)
- Towards Data Science - David Moore: <https://towardsdatascience.com/fastapi-versus-flask-3f90adc9572f/>
(Letzter Aufruf: 04.12.2025 13:16)
- Wikipedia: [https://en.wikipedia.org/wiki/Django_\(web_framework\)](https://en.wikipedia.org/wiki/Django_(web_framework))
(Letzter Aufruf: 05.01.2026 11:18)
- Wikipedia: https://en.wikipedia.org/wiki/Django_Software_Foundation
(Letzter Aufruf: 04.12.2025 13:11)
- Wikipedia: <https://en.wikipedia.org/wiki/FastAPI>
(Letzter Aufruf: 05.01.2026 11:17)
- Wikipedia: [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework))
(Letzter Aufruf: 05.01.2026 11:18)

7. Hilfsmittel

- LaTeX-Vorlage: Pascal Michailat - Minimalist LaTeX Template for Academic Papers:
<https://github.com/pmichailat/latex-paper>
- Comprehensive TeX Archive Network - The CTAN <http://ctan.org>
- Integrated Development Environment: Visual Sutdio Code 1.106
- Betriebssysteme: Windows 10 22H2, Windows 11 24H2, Ubuntu 24.04
- Genutzte LLMs: Google Gemini 2.5 Flash, Google Gemini 3 Pro
- dict.leo.org
- openthesaurus.de
- korrekturen.de
- silbentrennung24.de
- scholar.google.com
- Korrekturlesende: